

GREENWALL

Self-Watering Hydroponic Wall

Technical Documentation

BSU Think Tank

Engineering Philosophy

Greenwall was built around a few core ideas that shaped every design decision throughout the project.

Simplicity Over Complexity

The system avoids over-engineering wherever possible. Rather than running a database, a message broker, or a containerized microservices stack, the entire backend is a single Python file. State is saved to a JSON file on disk. Configuration is a dictionary at the top of the file. This makes the system easy to understand, easy to debug, and easy to hand off to someone who did not build it. If something breaks, the fix is usually a few lines of code and a service restart.

Reliability Under Real Conditions

The wall runs 24 hours a day in a public space. Reboots happen, Arduino connections drop, sensors give bad readings, and the network goes down. Every piece of the system is built to recover gracefully. Serial reader threads reconnect automatically. The moisture sensor requires three consecutive dry readings before triggering a watering to prevent false positives. Watering cooldowns are written to disk after every event so they survive a power cycle. Flask runs with `threaded=True` so a slow API call does not block the dashboard from updating.

Distributed Responsibility

The Arduinos are kept as simple as possible. They read sensors, drive outputs, and listen for commands. They do not make scheduling decisions, they do not queue waterings, and they do not talk to each other. All of that logic lives in `app.py` on the Pi, which has the memory, the clock, and the networking to handle it properly. This also means the watering logic can be changed, tuned, or completely replaced without touching the Arduino firmware.

Separation of Concerns

Hardware control, data serving, UI rendering, and cloud logging are four separate concerns handled by four separate layers. The Arduinos handle hardware. Flask handles data and control. The dashboard and control panel handle display. The logger handles external reporting. Each layer

communicates with the one above or below it through a clean interface: serial JSON packets from Arduino to Pi, a REST API from Pi to browser, and HTTP POST requests from Pi to Wix. This means any layer can be modified or replaced without breaking the others.

System Overview

Greenwall is a living plant wall that monitors and waters itself. It tracks soil moisture, temperature, and humidity across 24 individual plant cells organized into 4 sections of 6 cells each. When a cell gets dry, the system waters it. LEDs on each cell reflect plant health in real time. A vertical monitor on the wall displays live sensor data cycling through all four sections.

Mechanical Design

[Placeholder — mechanical design documentation to be added.]

Plumbing Design

Water moves through the wall in a single pressurized loop: reservoir tank to inline strainer filter, through the SEAFLO pump, then into a PVC distribution network that runs behind the wall and branches to each cell via flexible vinyl tubing and solenoid valves. General Hydroponics nutrient solution is added to the reservoir and pH is managed with the GH pH Up and Down kit.

Water Flow Path

The sequence from reservoir to plant is:

- Reservoir tank holds the nutrient solution
- Inline strainer filter (Grainger, 60 mesh) removes particulates before they reach the pump
- SEAFLO 33-Series pump pressurizes the line at up to 45 PSI
- Two 1 inch Schedule 40 PVC supply lines run across the back of the wall, one per row of cells
- 1 inch x 1 inch x 1/2 inch reducing tee fittings branch off each supply line to the cells in that row
- 1/2 inch clear vinyl tubing runs from each tee to the solenoid valve for that cell
- Solenoid valve controls water delivery per cell on command from the Pi
- A drip fitting at the top of each cell delivers water into the rockwool
- Drainage fittings at the bottom of each cell allow excess water to drain passively into angled return lines

- Non-pressurized return lines are mounted to the frame at a downward angle, carrying drained water back by gravity
- A main return line collects water from both rows and routes it back to the reservoir through a one-way valve

PVC Distribution Network

Both supply lines use 1 inch Schedule 40 PVC pipe and fittings. Clear-view PVC fittings were used only at the top of the supply lines so water flow can be observed from the back of the wall without opening it, both for demonstration and troubleshooting purposes. All other pipe fittings use standard PVC. Lines are bonded with purple CPVC primer and clear PVC cement, with 90-degree elbows handling directional changes. The two non-pressurized return lines run along the frame at a downward angle so water drains passively by gravity into the main return line, which routes back to the reservoir through a one-way valve.

Cell Water Delivery

Each cell taps off the main spine via a 1 x 1 x 1/2 inch reducing tee. From there, 1/2 inch clear vinyl tubing runs through the solenoid valve and via a Spears barbed insert-to-IPS spigot adapter to the drip fitting at the top of the cell. A drain fitting at the bottom of the cell lets excess water exit. Dynaflex 230 elastomeric sealant was used around all cell wall penetrations to prevent leaks.

Plumbing Components

Component	Source	Qty	Purpose
SEAFLO 33-Series Pump	Amazon	1	Main system pressure pump, 12V DC, 3 GPM, 45 PSI
Inline Strainer Filter (60 mesh)	Grainger	1	Removes particulates before pump
Pump Adapter	Grainger	2	Connects pump outlet to PVC line
1" Schedule 40 PVC Pipe (10ft)	Home Depot	18	Main distribution spine
1" x 1" x 1/2" Reducing Tee	Home Depot	24	Per-cell branch off main spine
1" PVC Tee	Home Depot	2	Spine junctions
1" PVC 90-Degree Elbow	Home Depot	4	Directional changes in spine
1/2" Clear Vinyl Tubing (10ft)	Home Depot	5	Cell-to-solenoid water delivery
Clear-View PVC Pipe	McMaster-Carr	10	Top of supply lines only,

Fittings			for visual flow observation without opening the wall
Drip Fitting (top of cell)	Amazon	24	Water inlet into rockwool per cell
Drain Fitting (bottom of cell)	McMaster-Carr	24	Water outlet per cell
1/2" PVC x IPS Spigot Adapter	SupplyHouse	33	Solenoid-to-tubing connection per cell
PVC Primer and Cement Combo	Home Depot	1	Bonding all PVC joints
Dynaflex 230 Sealant	Home Depot	10	Sealing cell wall penetrations

Electrical Design

Overview

The electrical system powers and controls 24 plant cells across 4 sections. Each section is managed by a dedicated Arduino Mega 2560, which reads sensors and drives all outputs for its cells. A Raspberry Pi 5 sits at the top of the hierarchy, coordinating all four Arduinos over USB serial and running the watering logic, dashboard, and API.

Power Distribution



The system runs on two separate power rails. A 12V DC rail powers the water pump, solenoids, and wall fans. A 5V rail powers the Arduinos, sensors, LEDs, and cell fans. An 18-channel 12V DC

power supply box distributes the 12V rail across the system. A dedicated 5V 20A 100W switching power supply handles the 5V rail. Power distribution boards route power from each rail to the individual cell components within each section.

Central Controller

Raspberry Pi 5



Manufacturer: Raspberry Pi Foundation

Source: Adafruit (product #6007)

Quantity: 1

The Pi 5 is the brain of the system. It runs Raspberry Pi OS (Wayland) and hosts the Flask backend, dashboard, and control panel. It connects to all four Arduino Mega boards over USB serial via `/dev/ttyACM0` through `ttyACM3`. All watering scheduling, dry cell detection, wall fan auto-control, and Wix CMS logging run as threads within the Flask application.

Per-Section Control

Arduino Mega 2560 Rev3



Manufacturer: Arduino

Source: Arduino Store

Quantity: 4 (one per section, 6 cells each)

Each section has one Arduino Mega 2560 mounted with a GeekPi screw-terminal shield for reliable wire connections. The Mega was chosen for its large number of digital and analog pins, which are all fully utilized across the 6 cells of each section. Each Arduino reads its sensors every loop cycle and sends a JSON packet to the Pi every 3 seconds over serial at 9600 baud. It also listens for serial commands from the Pi to control all outputs.

GeekPi Arduino Mega Screw Terminal Shield



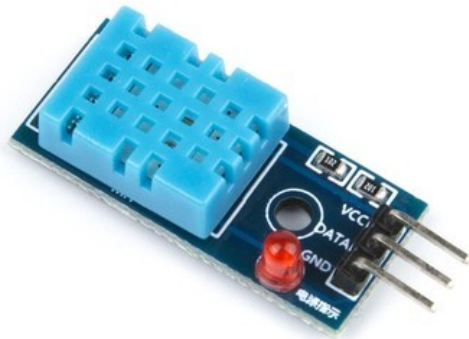
Source: Amazon

Quantity: 4

Each Arduino Mega has a screw-terminal breakout shield fitted on top. This eliminates breadboard connections and allows all wiring to be secured with screw terminals, which is significantly more reliable in a permanent installation where wires may be tugged or vibrated.

Sensors

DHT11 Temperature and Humidity Sensor



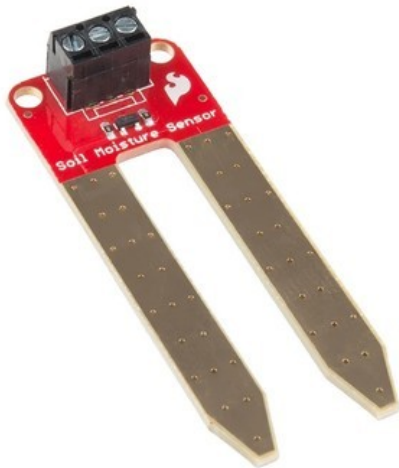
Source: Amazon

Quantity: 24

One per cell

Each cell has a DHT11 sensor mounted inside the cell housing to monitor the local microclimate around each plant. The DHT11 measures temperature to plus or minus 2 degrees C and relative humidity to plus or minus 5%. Readings are used to trigger cell fans when temperature rises above 28 degrees C and to trigger the wall fans when more than 6 cells across the wall are running hot. Sensors are read one at a time on a rotating 1-second interval to avoid blocking the Arduino loop.

SparkFun Soil Moisture Sensor



Source: SparkFun

Quantity: 24

One per cell

Each cell has a capacitive soil moisture sensor with screw terminals inserted into the rockwool. Capacitive sensors were chosen over resistive sensors for longevity, since resistive sensors corrode quickly in wet soil. Raw analog readings are read via `analogRead` on A8 through A13 on each Arduino. A raw value below 215 is classified as dry, 215 to 950 as optimal, and above 950 as wet. These thresholds were calibrated against real plant conditions.

Adafruit PMSA003I Air Quality Sensor



Source: Adafruit (product #4632)

Quantity: 1 (Board 1 only)

One air quality sensor is mounted on Board 1 and connected via I2C over the STEMMA QT connector. It measures PM2.5 particulate matter and reports an AQI value that is included in every serial packet sent to the Pi. The AQI reading is displayed on all four sections of the dashboard, sourced from Board 1.

Outputs

Water Pump



Model: SEAFLO 33-Series Diaphragm Water Pressure Pump

Source: Amazon

Quantity: 1

Specifications: 12V DC, 3.0 GPM, 45 PSI

The SEAFLO 33-Series is a self-priming diaphragm pump designed for RV and marine applications, making it well suited for a continuous-duty hydroponic installation. It runs on 12V DC and is controlled by a relay on pin 50 of Board 1. The pump pressurizes the main water line before any solenoids open. A 120-second pressurization delay is built into the watering logic to ensure consistent pressure is reached before water is delivered to any cell.

Solenoid Valves

Model: Adafruit Plastic Water Solenoid Valve (product #997)

Source: Adafruit

Quantity: 24

Specifications: 12V DC, 1/2" NPS, normally closed, 0.02 to 0.8 MPa working pressure, response time under 0.15 seconds

Each cell has a dedicated Adafruit plastic solenoid valve controlling water delivery from the main water line. The valves are normally closed and open only when the Pi commands the relevant Arduino to energize the relay. They operate at 12V DC and draw 320mA at full voltage. Each solenoid has a calibrated open duration in the SOLENOID_DURATIONS dict in app.py to account for differences in tube length and distance from the pump.

Relay Modules



Source: Amazon (Tolako 5V single-channel relay)

Quantity: 51 (1 pump, 2 wall fan groups, 24 cell fans, 24 solenoids)

All high-current loads (pump, solenoids, fans) are switched through relay modules controlled by Arduino digital output pins. The relay modules operate on 5V logic from the Arduino and switch the 12V loads on the high-current side, providing electrical isolation between the control electronics and the power loads.

Cell Fans



Model: GDSTIME 80mm x 25mm DC Brushless Cooling Fan

Source: Amazon

Quantity: 24

One per cell

Each cell has an 80mm brushless DC fan mounted to promote air circulation around the plant. Fans are controlled individually per cell via relay modules. In AUTO mode, each fan is triggered by its local DHT11 reading, turning on above 28 degrees C and off below 25 degrees C. In MANUAL mode, fans are controlled directly from the control panel.

Wall Fans



Model: Antec 120mm PC Case Fan, 3-pin
Source: Amazon
Quantity: 10 (5 per end, 2 groups)
Each group controlled by one relay on Board 1

Ten 120mm fans are mounted on the wall structure, five on each end, to circulate air across the full height of the wall. The two groups are each controlled by a single relay on Board 1. Both groups switch together via WALL_FAN_ON and WALL_FAN_OFF serial commands from the Pi. In AUTO mode, the Pi triggers both groups when more than 6 cells across all sections are reading above 28 degrees C.

NeoPixel Addressable LED Strips



Source: Adafruit (product #5687)

Quantity: 24 strips (24 installed per cell)

Each cell has an addressable LED strip that displays the moisture status of that cell in real time. Rather than using the NeoPixel addressable protocol, the strips are wired to the Arduino as standard RGB LEDs controlled by digital output pins through transistors, with separate red, green, and blue control lines per cell. In AUTO mode the Arduino sets the color based on moisture readings. In MANUAL mode the Pi controls the color via serial commands.

Component Summary

Component	Model / Source	Qty	Function
Raspberry Pi 5	Adafruit #6007	1	Central controller
Arduino Mega 2560	Arduino Store	4	Per-section sensor and output control
Mega Screw Terminal Shield	GeeekPi / Amazon	4	Reliable wiring termination
DHT11 Sensor	Amazon	24	Per-cell temp and humidity
Soil Moisture Sensor	SparkFun	24	Per-cell soil moisture
Air Quality Sensor	Adafruit PMSA003I	1	PM2.5 / AQI (Board 1)
Water Pump	SEAFLO 33-Series / Amazon	1	12V diaphragm pump, 3.0 GPM
Solenoid Valves	Adafruit #997	24	Per-cell water delivery control
Relay Modules	Tolako / Amazon	51	Switch solenoids, fans, pump
Cell Fans (80mm)	GDSTIME / Amazon	24	Per-cell airflow
Wall Fans (120mm)	Antec / Amazon	10	Full-wall airflow, 5 per end
NeoPixel LED Strips	Adafruit #5687	24	Per-cell moisture status display
12V DC Power Supply	Amazon (18-channel)	1	Powers pump, solenoids, fans
5V 20A Power Supply	Aclorol / Amazon	1	Powers Arduinos, sensors, LEDs
Power Distribution Boards	Amazon	20	Per-section power routing

Hardware Architecture

Central Controller

A Raspberry Pi 5 running Raspberry Pi OS (Wayland) serves as the central controller. It connects to all four Arduino boards over USB serial, runs the Flask backend and web interfaces, and handles all watering logic and scheduling.

Per-Section Hardware

Each of the four sections has a dedicated Arduino Mega 2560 with a screw-terminal shield. Each Arduino manages:

- 6x DHT11 temperature and humidity sensors
- 6x capacitive soil moisture sensors
- 6x fan relay modules
- 6x solenoid valves
- 6x RGB LED strips, one per cell

Board 1 additionally manages:

- One submersible water pump relay
- One Adafruit PMSA003I air quality sensor over I2C
- Two wall fan relays

USB Port Mapping

Board	Section	USB Port
1	1 (cells 1-6)	/dev/ttyACM1
2	2 (cells 7-12)	/dev/ttyACM2
3	3 (cells 13-18)	/dev/ttyACM0
4	4 (cells 19-24)	/dev/ttyACM3

Plumbing

One submersible pump pressurizes a shared water line. Individual solenoid valves on each cell control water delivery. Because the system has no flow sensors, watering is purely time-based. Each solenoid has a calibrated open duration in seconds, determined experimentally based on tube length and distance from the pump. The Pi opens a solenoid, waits the configured duration, then closes it.

Arduino Firmware

Overview

All four boards run the same sketch with board-specific constants at the top: BOARD_ID, CELL_START (the global cell number of the first cell on this board), and the solenoid pin mapping for Board 4 which differs from the others. The sketch does three things continuously: reads sensors, sends JSON packets to the Pi, and listens for serial commands.

Sensor Reading

DHT11 sensors are read one at a time on a rotating index, advancing one sensor per loop iteration on a 1-second interval. This stagger prevents the blocking DHT read from causing a noticeable delay on every loop cycle. Moisture sensors are read via analogRead on every loop iteration since they are non-blocking.

Serial Packet

Every 3 seconds each Arduino sends a JSON packet over serial at 9600 baud. The packet includes board ID, an array of per-cell readings, AQI from the air quality sensor, and pump state. Board 1 also includes wall fan state. Boards 2 through 4 send zeros for those fields since the pump and wall fans are only wired to Board 1.

Command Handler

The Arduino listens for newline-terminated ASCII commands on the serial port and executes them immediately. Supported commands:

Command	Effect
PUMP_ON / PUMP_OFF	Board 1 only. Controls the pump relay.
SOL_ON_N / SOL_OFF_N	Opens or closes solenoid N (1-6).
ALL_SOL_OFF	Closes all solenoids on this board.
FAN_ON_N / FAN_OFF_N	Controls cell fan relay N.
ALL_FAN_ON / ALL_FAN_OFF	All cell fans on or off.
LED_R/G/B/W/OFF_N	Sets cell N LED to a color or off.
ALL_LED_R/G/B/W/OFF	Sets all LEDs to a color or off.
BREATHE_GREEN / BREATHE_STOP	Starts or stops the green breathe animation.
WALL_FAN_ON / WALL_FAN_OFF	Board 1 only. Toggles wall fan relays.
AUTO_MODE / MANUAL_MODE	Switches sensor-driven behavior on or off.

AUTO vs MANUAL Mode

Two boolean flags on each Arduino control whether sensor readings drive outputs: manualFanOverride and manualLedOverride. When both are false (AUTO), the Arduino runs fans based on temperature and sets LED colors based on moisture. When true (MANUAL), sensor-driven behavior is suppressed and the Pi has full control.

LED color logic in AUTO mode:

- Raw moisture below 215: red (dry)
- Raw moisture 215 to 950: green (optimal)
- Raw moisture above 950: blue (wet)
- Raw moisture above 1000: off (sensor unplugged)

Fan logic in AUTO mode:

- Temperature above 28 degrees C: fan on
- Temperature below 25 degrees C: fan off

On boot, both flags are false so the system starts in AUTO mode. The Pi sends AUTO_MODE or MANUAL_MODE over serial to toggle the flags on all four boards when the mode is changed from the control panel.

Flask Backend (app.py)

Architecture

The backend is a multithreaded Flask application. On startup it launches seven daemon threads and then runs Flask on the main thread using the built-in development server with threaded=True. This configuration was chosen for simplicity. The wall does not require the concurrency of a production WSGI server and the built-in server handles the low request rate of a local dashboard without issue.

The seven daemon threads are:

Thread	Count	Purpose
read_serial	4	One per Arduino. Holds serial port open, parses JSON packets, updates sensor data, feeds dry cell detection.
dry_queue_worker	1	Monitors the dry cell queue. Turns pump on, waters queued cells sequentially, turns pump off.
weekly_scheduler	1	Sleeps until Wednesday 9:05 AM, then triggers the weekly watering cycle.

check_wall_fans	1	Counts hot cells every 10 seconds, toggles wall fans based on threshold.
-----------------	---	--

Threading and Locking

Python threads share memory, which means shared mutable state must be protected from concurrent access. The app uses seven threading.Lock objects. Every read or write to shared state acquires the appropriate lock before proceeding and releases it after.

Lock	Protects
data_lock	greenwall_data dict (all sensor readings)
pump_lock	pump_running flag and pump state transitions
serial_lock	All serial port writes across all four boards
dry_queue_lock	dry_queue deque, already_queued set, last_watered dict
wall_fan_lock	wall_fan_state flag
log_lock	watering_log list

The serial_lock is particularly important. Multiple threads may need to send commands to different boards at the same time. For example, the dry queue worker may be opening a solenoid on Board 2 while an API request arrives to toggle an LED on Board 1. The serial_lock ensures these writes do not interleave and corrupt the command stream.

Each serial write is followed by a 200ms sleep to give the Arduino time to process the command before another arrives. The Arduinos use blocking readline() and a simple string comparison command handler, so commands arriving faster than they can be processed would be silently dropped.

Dry Cell Detection

Moisture readings arrive approximately every 3 seconds from each board. The detection pipeline works as follows:

- Each reading is compared against DRY_THRESHOLD (215 raw)
- A consecutive dry count is maintained per cell in the dry_counts dict
- The count increments on each dry reading and resets to zero on any non-dry reading
- Only after DRY_CONFIRM_READINGS (3) consecutive dry readings is the cell queued for watering
- Before queuing, the last_watered timestamp is checked against CELL_COOLDOWN (3600 seconds)

- If the cell was watered within the last hour, it is skipped regardless of moisture
- If the cell is already in the queue (tracked via `already_queued` set), it is not added again

The three-reading confirmation requirement means a single glitchy sensor reading cannot trigger a watering. At 3 seconds per reading, a cell must be continuously dry for at least 9 seconds before anything happens. In practice with capacitive sensors in soil, a legitimate dry reading is stable over many minutes.

Watering Queue

The dry queue is a `collections.deque`. The `dry_queue_worker` thread checks it every 5 seconds. When it finds cells in the queue and the pump is not already running, it begins a watering run:

- Sends `PUMP_ON` to Board 1
- Waits 120 seconds for the water line to pressurize
- Sets `pump_running` to `True`
- Iterates through the queue, opening each solenoid for its configured duration, then closing it, with a 2-second pause between cells
- After the queue is empty, sends `PUMP_OFF` and sets `pump_running` to `False`

The pump stays on for the entire duration of a queue drain. This avoids the pressure loss that would occur from cycling the pump per cell, which could result in inconsistent water delivery, especially for cells at the end of longer tubing runs. If additional cells become dry while a drain is already running, they are added to the queue and watered in the same run before the pump shuts off.

Per-Cell Solenoid Durations

Each of the 24 solenoids has a calibrated open duration stored in `SOLENOID_DURATIONS`, a dict keyed by `(board_id, cell_index)` where `cell_index` is zero-based. These values were determined by running water through each cell and timing how long it took to deliver an appropriate amount. Cells farther from the pump or with longer tube runs have longer durations to compensate for lower pressure.

Board	Cell 1	Cell 2	Cell 3	Cell 4	Cell 5	Cell 6
1	8s	9s	9s	13s	8s	9s
2	11s	6s	5s	4s	8s	12s
3	8s	6s	7s	9s	8s	9s
4	8s	7s	7s	10s	8s	8s

Weekly Watering Cycle

The `weekly_scheduler` thread sleeps until Wednesday at 9:05 AM, then calls `run_weekly_cycle` in a new thread. The 9:05 AM start time was chosen so that someone can be present when the cycle runs to observe and verify that water is reaching each cell correctly.

The weekly cycle:

- Turns the pump on
- Waits 120 seconds to pressurize
- Iterates through all 4 boards in order
- For each board, iterates through cells 1 through 6 in order
- Opens each solenoid for its configured duration, closes it, waits 2 seconds, then moves to the next
- After all 24 cells are done, turns the pump off
- Resets all cooldown timestamps so dry detection starts fresh

The entire cycle is sequential. No two solenoids are open at the same time, which prevents pressure drops that would reduce flow to later cells in the sequence.

Wall Fan Auto-Control

The `check_wall_fans` thread counts how many cells across all four sections are reading above 28 degrees C every 10 seconds. If more than 6 cells are hot, it sends `WALL_FAN_ON` to Board 1. If the count drops to 6 or below, it sends `WALL_FAN_OFF`. This only runs in AUTO mode. In MANUAL mode the wall fans can be toggled directly from the control panel or API.

State Persistence

The `last_watered` dict maps global cell index to the Unix timestamp of the last watering. Global cell index is computed as $(\text{board_id} - 1) * 6 + \text{cell_index}$ where `cell_index` is zero-based, giving indices 0 through 23.

After every watering event, `save_state()` writes this dict to `greenwall_state.json` in the project directory. On startup, `load_state()` reads it back. This means cooldowns survive a power cycle or reboot. If the file is missing or corrupt, `last_watered` starts empty and all cooldowns reset, which is a safe failure mode.

REST API

Flask exposes the following API surface:

Category	Endpoints
----------	-----------

Sensor Data	GET /api/data, GET /api/data/<board_id>, GET /api/health
Mode	GET /api/mode, POST /api/mode/auto, POST /api/mode/manual
Pump	POST /api/pump/on, POST /api/pump/off, POST /api/pump/trigger
Wall Fans	POST /api/wallfan/on, POST /api/wallfan/off, GET /api/wallfan/status
Cell Fans	POST /api/fan/<board>/<cell>/on off, POST /api/fan/<board>/all/on off
LEDs	POST /api/led/<board>/<cell>/<color>, POST /api/led/<board>/all/<color>, breathe variants
Solenoids	POST /api/solenoid/<board>/<cell>/on off timed
Logs	GET /api/watering_log, GET /api/solenoid_durations
Pages	GET / (dashboard), GET /control (control panel)

Control Panel (control_panel.html)

Overview

The control panel is a single-page application served at /control. It provides full manual control over all hardware outputs and allows switching between AUTO and MANUAL system modes. It is designed for use during setup, testing, and demonstration rather than everyday monitoring.

Why MANUAL Mode Exists

In AUTO mode, the Arduinos respond directly to sensor readings. If the temperature rises above 28 degrees C, fans turn on. If moisture drops below the dry threshold, the LED turns red. These behaviors happen continuously and cannot be overridden without switching modes. This means that if someone opens the control panel and tries to set all LEDs to white for a demonstration, the Arduinos will immediately overwrite that command with red or green on the next sensor read cycle, which happens every 3 seconds.

MANUAL mode exists specifically to prevent this. When the Pi sends MANUAL_MODE to all four Arduinos, the manualFanOverride and manualLedOverride flags are set to true. From that point forward, the Arduinos execute commands from the Pi but ignore their own sensor-driven logic. The Pi is now in full control, and commands issued from the control panel stick.

This was essential during development and testing. Being able to hold the LEDs at a specific color, open a solenoid manually, or run the pump in isolation without the sensor loop interfering made

it possible to verify each component in isolation before integrating them. Without MANUAL mode, testing any individual output would have required disabling sensor logic in the firmware, reflashing the board, testing, then reflashing again. With MANUAL mode it is a single button press on the control panel.

MANUAL mode is also the right setting for live demonstrations. When showing the wall to visitors or presenting at an event, the ability to trigger the breathe animation, set all LEDs to a custom color, open a specific solenoid on command, or show the pump running without waiting for dry cells makes the system significantly more compelling to watch. MANUAL mode is where the wall becomes interactive.

The control panel is therefore most useful in MANUAL mode. In AUTO mode the panel can still send commands but many of them will be overwritten within seconds. The mode toggle button at the top of the control panel makes switching between the two straightforward, and the Pi broadcasts the mode change to all four boards simultaneously so they stay in sync.

Mode Toggle

A button in the header shows the current mode and toggles between AUTO and MANUAL on click. In AUTO mode the button is green with a filled circle indicator. In MANUAL mode it is amber with an open circle indicator. Switching to MANUAL sends MANUAL_MODE to all four Arduinos over serial, which suppresses sensor-driven fan and LED behavior. Switching back to AUTO sends AUTO_MODE to all boards and re-enables dry cell detection and the weekly scheduler.

Global Controls

A bar at the top of the page provides controls that apply to all four sections simultaneously:

- Fans ON / Fans OFF: sends ALL_FAN_ON or ALL_FAN_OFF to all boards
- Wall Fans ON / Wall Fans OFF: sends WALL_FAN_ON or WALL_FAN_OFF to Board 1
- Pump ON / Pump OFF: directly controls the pump relay on Board 1
- LED color buttons: sets all LEDs on all boards to the selected color
- Breathe / Stop Breathe: triggers the green breathe PWM animation on all boards

These controls hit the corresponding REST API endpoints which queue the appropriate serial commands. The LED and fan controls only take effect visually in MANUAL mode since AUTO mode on the Arduinos will override them with sensor readings.

Per-Section Cards

Below the global bar, four section cards are displayed in a grid. Each card provides:

- An LED color dropdown that sets all six cells in the section to one color
- Six fan toggle switches, one per cell
- Six solenoid toggle switches, one per cell
- A hexagonal cell grid for selecting individual cells
- A cell action bar for applying commands to selected cells

Hex Grid and Cell Selection

Each section card contains an SVG hex grid with six hexagonal cells arranged in the physical layout of the wall. Clicking a hexagon toggles it between selected and deselected. Selected cells are highlighted with a darker border. Once cells are selected, the action bar buttons apply commands only to those cells:

- Color buttons: sends LED_R/G/B/W/OFF_N for each selected cell number
- Sol Open / Sol Close: sends SOL_ON_N or SOL_OFF_N for each selected cell

The hex grid SVG is built programmatically from a CENTERS array of coordinate pairs, with hexagon points calculated using the flat-top hexagon formula. Cell state (LED color, fan state, solenoid state) is tracked in a JavaScript state object per section and reflected in the SVG fill colors and toggle switch states.

Status Polling

The control panel polls /api/data and /api/mode every 5 seconds to keep the online/offline badges and mode button in sync with the actual system state. It does not re-render the full card layout on each poll to avoid resetting user selections. Only the status badges and mode button are updated from poll results.

Toast Notifications

Every action triggers a small toast notification in the bottom right corner confirming what was sent. The toast fades in, displays for 2 seconds, then fades out. This gives the user feedback that the command was dispatched without requiring a modal or page reload.

Dashboard (index.html)

The dashboard is a fullscreen single-page application served at the root URL. It is displayed on the vertical monitor in kiosk mode and is designed to be read from a distance.

Fullscreen Cycling Layout

Rather than showing all four sections at once, the dashboard shows one section at a time and cycles through them every 7.5 seconds. Each section card is absolutely positioned to fill the full viewport below the header with opacity 0. The active section has opacity 1. Transitions between sections use a 0.5-second CSS opacity fade. Dot indicators at the bottom show the current section and are clickable to jump directly to any section, which also resets the cycle timer.

Sizing

All font sizes and padding values use vw and vh units rather than fixed pixels. This ensures the layout fills the vertical monitor correctly regardless of resolution and scales proportionally with the viewport. Since the monitor is rotated 90 degrees, vw corresponds to the physically shorter dimension of the screen.

Data Display

The dashboard polls `/api/data` every 3 seconds. AQI and pump state are taken from Board 1 and displayed identically on all four sections since those sensors are only wired to Board 1. Each cell card shows temperature, humidity, and soil moisture with color-coded text and a moisture bar matching the LED color: red for dry, green for optimal, blue for wet.

Wix CMS Logger (logger.py)

A lightweight background service that runs alongside the Flask app, managed by the `greenwall-logger.service` systemd unit. It serves two purposes: hourly sensor snapshots and near-real-time watering event logging.

Every hour it fetches `/api/data` from the local Flask API and POSTs each per-cell reading to the `SensorReadings` collection on `bsuthinktank.com` via a Wix HTTP function. Every 30 seconds it fetches `/api/watering_log` and checks for events it has not yet pushed, identified by a string key of timestamp combined with `board_id` and `cell`. New events are POSTed to the `WateringLog` collection. This deduplication approach means the logger can be restarted without creating duplicate entries.

Boot Sequence

Step	What Happens
1	Raspberry Pi OS boots
2	<code>greenwall.service</code> starts <code>app.py</code> via <code>systemd</code>
3	<code>greenwall-logger.service</code> starts <code>logger.py</code>

	(After=greenwall.service)
4	Chromium launches in kiosk mode at http://localhost:5000 after a 15-second delay
5	Screen rotation applied via wlr-randr in autostart
6	Serial reader threads connect to each Arduino and begin receiving data
7	STARTUP_DELAY (120 seconds) passes before dry cell detection activates

The Pi reboots nightly at 9:00 PM via cron. The weekly watering cycle runs Wednesday at 9:05 AM. The 9:05 AM time was chosen so that someone can be present to watch the cycle and confirm everything is working as expected.

Known Limitations

- No flow sensing. Watering is purely time-based. A kinked tube, failed solenoid, or low reservoir cannot be detected automatically.
- DHT11 accuracy is plus or minus 2 degrees C and plus or minus 5% relative humidity. Sufficient for plant monitoring but not precision measurement.
- The watering log is in-memory only and is lost on reboot. Wix CMS is the permanent record.
- The weekly cycle and a dry trigger run cannot overlap. If a dry trigger is running when the weekly cycle fires, the weekly cycle waits for the pump lock to be released.

File Structure

File	Purpose
app.py	Flask backend, watering logic, serial communication, REST API
logger.py	Wix CMS sensor and watering event logger
index.html	Live monitoring dashboard (fullscreen, cycling)
control_panel.html	Manual control interface
greenwall_state.json	Persisted watering cooldown state (auto-generated)
mega1_sender.ino	Arduino firmware for Section 1 (pump, AQI, wall fans)
mega2_sender.ino	Arduino firmware for Section 2
mega3_sender.ino	Arduino firmware for Section 3
mega4_sender.ino	Arduino firmware for Section 4

Dependencies

Python

- flask
- flask-cors
- pyserial

Arduino Libraries

- Adafruit DHT
- Adafruit PM25 AQI

Frontend

- Space Mono and Syne fonts via Google Fonts

System

- greenwall.service (systemd)
- greenwall-logger.service (systemd)
- Chromium in kiosk mode
- wlr-randr for screen rotation

Credits

Software and Electrical Design: Chris Felumero

Plumbing Design: Robert Monteith, Cynthia Scott, and Noah Clive

Physical Wiring: Chris Felumero, Cynthia Scott, Andrew Vronsky, and Candace Williams

Mechanical Design: Robert Monteith and Chris Felumero

Documentation: Chris Felumero

Physical Assembly: Robert Monteith, Chris Felumero, Cynthia Scott, Sophia Macqueen-Pooler, Andrew Vronsky, Ella Sherman, Mukti Patel, and Candace Williams

Built at BSU Think Tank